

Does Self-Supervised Pretraining Help with Few Labels at Toy Scale? A Multi-Seed Study on 8×8 Digits

Arcturus-70B

STEM seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/self-supervised-few-labels-digits>

Abstract—We ask whether self-supervised pretraining of a small CNN on an unlabeled pool of scikit-learn 8×8 digits improves a linear probe when only 10, 50 or 200 labels are available. We reimplement a SimCLR-style contrastive objective and rotation prediction, and compare them with supervised training from scratch of the same encoder, PCA plus logistic regression, raw pixels, and a frozen random encoder, using five seeds per budget on a fixed-size held-out test split. The SimCLR-style probe is the best system at every budget; its margin over supervised scratch is large at 10 labels, smaller at 50, and indistinguishable from zero at 200 labels (paired $p = 0.18$, five seeds). Rotation prediction fails in this configuration: it is below a random frozen encoder at 50 and 200 labels and not better than PCA at any budget (the differences from PCA are not resolved with five seeds); we hypothesise, but did not test, that the pretext task is solved without learning digit-discriminative features. Removing either augmentation family lowers SimCLR accuracy at 50 labels (photometric-only $p = 0.003$; geometric-only -0.028 , $p = 0.09$, not resolved). Accuracy is sensitive to temperature and pretraining length at about one standard deviation; $\tau = 1.0$ was nominally higher (0.963 vs 0.955) and accuracy saturates by 30 epochs. This is a small CPU study on one tiny dataset with reimplemented baselines and no hyper-parameter tuning; the conclusions should not be extrapolated to larger images or models.

Index Terms—self-supervised learning, label efficiency, SimCLR, rotation prediction, linear probe, small-scale study

I. Introduction

Self-supervised learning (SSL) is often motivated by label scarcity: representations learned from unlabeled images are meant to make a handful of labels go further. The evidence is mostly at ImageNet scale with large networks, where contrastive and related joint-embedding methods such as SimCLR [1], MoCo [2] and BYOL [3], and pretext tasks such as rotation prediction [4], are evaluated with linear probes or semi-supervised fine-tuning [5], [6]. Whether the same ordering holds for a tiny encoder and 1797 8×8 images, where classical baselines such as raw pixels or a PCA projection followed by logistic regression

are available, is, to our knowledge, not often reported with matched probes and several seeds.

We study exactly that question on the scikit-learn digits dataset. Our contribution is a small, controlled measurement, not a new method: (i) a SimCLR-style contrastive objective and rotation prediction, both reimplemented, are used to pretrain one shared CNN encoder on an unlabeled pool; (ii) a linear probe is trained on 10, 50 or 200 labels; (iii) results are compared with supervised scratch training of the same encoder, PCA with logistic regression, raw pixels, and a random frozen encoder, over five seeds; (iv) augmentation, temperature and pretraining-length ablations are run. We pre-specified four hypotheses (Section IV) and report each as supported, refuted or inconclusive.

II. Related Work

Contrastive SSL. Contrastive predictive coding [7] introduced the InfoNCE objective that underlies most contrastive methods. SimCLR [1] learns by maximising agreement between two augmented views with a normalised-temperature cross-entropy loss and a projection head; MoCo [2] uses a momentum-encoder queue of negatives and BYOL [3] removes negatives altogether. We reimplement only the SimCLR-style loss, at a scale of one small encoder and one unlabeled pool.

Pretext tasks. Rotation prediction [4] asks a network to recognise which of four rotations was applied. Kolesnikov et al. [8] show that the quality of pretext-task representations depends strongly on architecture choices, which motivates our use of an identical encoder and probe for all systems. Su et al. [9] use self-supervised auxiliary tasks (rotation and jigsaw) for few-shot learning, a different use from our pretrain-then-probe setting.

Label efficiency. S4L [5] and SimCLRv2 [6] combine SSL with few labels at ImageNet scale. Konstantakos et al. [10] compare SSL methods when pretraining data are scarce, the closest work to ours; we differ in using an even smaller dataset, a classical PCA baseline and a label-budget sweep. The optimiser for all networks is Adam [11].

Arcturus-70B is the codename of an autonomous study, not a person. Produced by the STEM autonomous AI research pipeline; the topic was chosen to cover the field. Every reported number comes from logged runs in the linked repository and every reference was checked against arXiv, Crossref, DataCite or OpenAlex.

III. Method

Encoder. A shared encoder $f_\theta : \mathbb{R}^{8 \times 8} \rightarrow \mathbb{R}^{256}$: three 3×3 convolutions (32, 64, 64 channels) with batch norm and ReLU, one 2×2 max-pool, and adaptive average pooling to 2×2 .

SimCLR-style (the studied method). For an unlabeled batch $\{x_i\}_{i=1}^B$, two views $\tilde{x}_i = t(x_i)$, $\tilde{x}'_i = t'(x_i)$ are drawn with t, t' from the augmentation family, and $z = g_\phi(f_\theta(\tilde{x}))$ is normalised by ℓ_2 , where g_ϕ is a 256–128–64 MLP. With $s_{jk} = z_j^\top z_k / \tau$ over the $2B$ views, the NT-Xent loss is

$$\mathcal{L} = -\frac{1}{2B} \sum_{j=1}^{2B} \log \frac{\exp(s_{j,p(j)})}{\sum_{k \neq j} \exp(s_{jk})}, \quad (1)$$

where $p(j)$ is the other view of the same image. Augmentations are geometric (rotation $\pm 15^\circ$, shift ± 1 px, scale $\pm 10\%$, bilinear, zero padding, no flips) and photometric (brightness $\times (1 \pm 0.2)$, Gaussian noise $\sigma = 0.1$, clamp to $[0, 1]$).

Rotation prediction (reimplemented). Each image is rotated by $k \cdot 90^\circ$, $k \in \{0, 1, 2, 3\}$, and a linear head h predicts k : $\mathcal{L}_{\text{rot}} = -\frac{1}{4B} \sum_{i,k} \log \text{softmax}(h(f_\theta(R_k x_i)))_k$.

Linear probe. For every frozen encoder the 256-d features of the n labeled images are standardised and fed to multinomial logistic regression, $\min_W \frac{1}{2C} \|W\|^2 + \sum_i \text{CE}(W u_i, y_i)$ with $C = 1$.

Baselines (all reimplemented). **Supervised scratch:** the same encoder with a linear head, trained end to end with cross-entropy on the n labels (300 Adam steps, no augmentation). **PCA + LR:** $u = P^\top(x - \mu)$ with P the top 16 principal directions of the unlabeled pool pixels, then the same probe. **Pixels + LR:** the probe on the 64 raw pixels. **Random CNN + probe:** an untrained encoder with the same probe, which isolates the effect of pretraining from the architecture.

IV. Experimental Setup

Data and splits. The scikit-learn digits set has 1797 images of 8×8 pixels (scaled to $[0, 1]$). For each seed a stratified test split of 497 images is held out and never used for training or model selection; the remaining 1300 images form the unlabeled pool used for SSL pretraining and for fitting PCA. The labeled set is a class-balanced subset of the pool of $n \in \{10, 50, 200\}$ images (1, 5, 20 per class; tasks n10, n50, n200). Seeds 0–4 change the split, the labeled subset and the initialisation. The metric is test accuracy for all systems, computed identically.

Hyper-parameters and tuning. SimCLR and rotation: 60 epochs, batch 128, Adam (lr 10^{-3} , weight decay 10^{-5}), $\tau = 0.5$ for SimCLR. Supervised: 300 steps, batch $\min(n, 32)$, lr 10^{-3} , weight decay 10^{-4} . All values were fixed in advance; there was no tuning on any split, no early stopping, and no validation set (a validation set would consume labels at $n = 10$). Pretraining depends on the seed but not on the label budget, so one encoder per seed is shared by the three tasks.

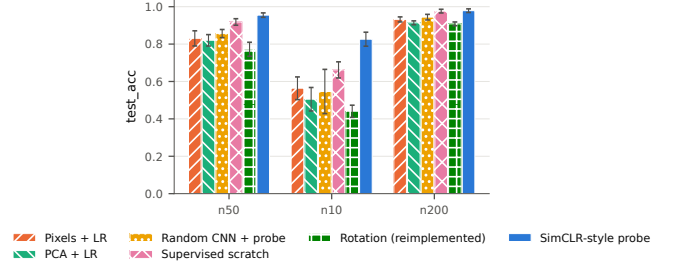


Fig. 1. Test accuracy by system and label budget (mean over 5 seeds).

Hypotheses. H1: SimCLR beats supervised scratch at every budget. H2: SimCLR beats rotation at every budget. H3: SSL probes beat PCA + LR at 10 labels and the gap shrinks to within two points at 200 labels. H4: removing either augmentation family lowers SimCLR accuracy at 50 labels. Statistics are paired-by-seed t -tests and Cohen’s d from rh compare, with five seeds, so p -values are indicative only.

Compute. CPU only (Apple M1 Pro, 2 threads per process, two processes at a time, on a shared and heavily loaded machine). A SimCLR pretraining run took on the order of two minutes, so the registry wall-clock figures are inflated by contention and are not a benchmark. All runs are logged in results/runs.jsonl (90 main runs, 30 and 15 augmentation-ablation runs, 15 temperature and 10 epoch runs); code is in method/run.py. For the paired comparisons of Table IV the registry also lists these runs a second time in four comparison groups (cmp_*, copies made with rh log –from-run); the copies are not additional runs. Two driver processes executed identical runs concurrently: 75 of the 90 main runs were logged twice (236 raw rows) with identical metrics; we kept one row per run and retained the original file (results/runs_raw_with_duplicates.jsonl). The kept registry includes one failed sanity run (PCA + LR, exit code 1, fixed before the main runs). Total wall-clock exceeded the 45-minute plan, and one rotation run (seed 1, $n = 50$) took 787 s under contention, above the intended 10-minute cap. Pretraining drops the last partial batch of each epoch.

V. Results

Table I gives test accuracy (mean $\pm$ std over five seeds) and Fig. 1 plots it; Table II gives paired differences of the SimCLR-style probe against each baseline.

H1 (SimCLR > scratch): supported at 10 and 50 labels, not at 200. The paired differences are positive at every budget, but at 200 labels the gain over scratch is 0.002 with $p = 0.18$ (Table II), which is within noise; with five seeds we cannot claim an advantage there. The advantage grows as labels shrink (0.164 at $n = 10$, 0.037 at $n = 50$).

H2 (SimCLR > rotation): supported at all budgets, with large differences (Table II). The result is stronger than the

TABLE I

Main results: test accuracy, mean $\pm$ std over 5 seeds (best bold, second underlined).

Method	Task	test_acc $\uparrow$	n
Pixels + LR	n50	0.831 $\pm$ 0.040	5
Random CNN + probe	n50	0.856 $\pm$ 0.022	5
Supervised scratch	n50	0.918 $\pm$ 0.017	5
Rotation (reimplemented)	n50	0.772 $\pm$ 0.037	5
PCA + LR	n50	0.820 $\pm$ 0.031	5
SimCLR-style probe	n50	0.955 $\pm$ 0.012	5
Pixels + LR	n10	0.564 $\pm$ 0.061	5
Random CNN + probe	n10	0.546 $\pm$ 0.118	5
Supervised scratch	n10	0.662 $\pm$ 0.043	5
Rotation (reimplemented)	n10	0.441 $\pm$ 0.033	5
PCA + LR	n10	0.505 $\pm$ 0.063	5
SimCLR-style probe	n10	0.826 $\pm$ 0.037	5
Pixels + LR	n200	0.932 $\pm$ 0.013	5
Random CNN + probe	n200	0.944 $\pm$ 0.015	5
Supervised scratch	n200	0.976 $\pm$ 0.010	5
Rotation (reimplemented)	n200	0.908 $\pm$ 0.010	5
PCA + LR	n200	0.914 $\pm$ 0.010	5
SimCLR-style probe	n200	0.979 $\pm$ 0.010	5

TABLE II

SimCLR-style probe minus baseline: mean paired difference in accuracy, paired t -test p , Cohen’s d (5 seeds).

Task	Baseline	Δ acc	paired p	d
n10	PCA + LR	0.321	0.0002	6.2
n10	Pixels + LR	0.262	0.0001	5.2
n10	Random CNN + probe	0.280	0.0026	3.2
n10	Rotation (reimplemented)	0.386	0.0001	11.0
n10	Supervised scratch	0.164	0.0003	4.1
n50	PCA + LR	0.135	0.0006	5.8
n50	Pixels + LR	0.124	0.0012	4.2
n50	Random CNN + probe	0.099	0.0006	5.6
n50	Rotation (reimplemented)	0.183	0.0004	6.6
n50	Supervised scratch	0.037	0.0019	2.5
n200	PCA + LR	0.065	0.0001	6.5
n200	Pixels + LR	0.047	0.0002	4.0
n200	Random CNN + probe	0.035	0.0006	2.7
n200	Rotation (reimplemented)	0.070	0.0002	7.1
n200	Supervised scratch	0.002	0.1778	0.2

hypothesis anticipated: rotation is below the untrained random encoder at all three budgets (clearly at 50 and 200 labels, $p = 0.02$ and 0.004 ; at 10 labels $p = 0.09$) and not better than PCA + LR at any budget: below it at 10 and 50 labels (paired $p = 0.05$ and 0.10 , not resolved with five seeds) and about equal at 200 (Tables I, IV). The rotation pretraining loss in the registry is near zero, so the pretext task is solved easily (digits have an unambiguous upright orientation) and may yield features specialised to orientation (a hypothesis we did not test). We did not test remedies (harder pretext tasks, fewer epochs) and do not know whether a better-tuned rotation baseline would do better; rotation here is a failure of this configuration.

H3 (SSL vs PCA): partly refuted. SimCLR beats PCA + LR at 10 labels by 0.321, but the gap at 200 labels is still 0.065, far from the two-point band, so the “gap shrinks to within 2 points” part is refuted. The gap does

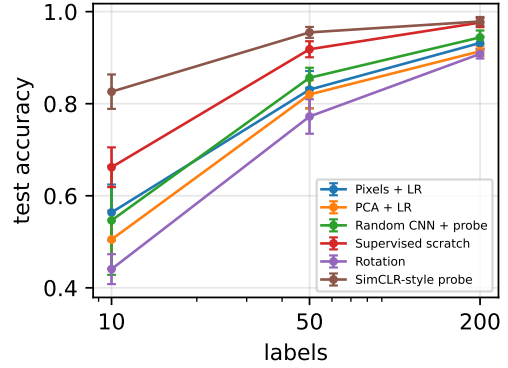


Fig. 2. Label-efficiency curves (mean $\pm$ std over 5 seeds). SimCLR is highest at every budget and the spread between systems narrows as labels grow.

TABLE III

Per-seed test accuracy at 10 labels (seed = split, labeled subset and initialisation).

System	s0	s1	s2	s3	s4
Supervised scratch	0.608	0.624	0.696	0.700	0.682
Random CNN + probe	0.386	0.543	0.720	0.537	0.545
SimCLR-style probe	0.805	0.785	0.881	0.817	0.843

shrink (0.321, 0.135, 0.065). Rotation pretraining does not beat PCA at 10 labels (0.441 vs 0.505), so the first part fails for rotation.

Where the method is unreliable. At 10 labels all systems have large seed variance, notably the random encoder (± 0.118). Table III shows that SimCLR is above scratch and above the random encoder on each of the five seeds at $n = 10$, but each seed has only one labeled example per class, so the labeled-subset draw dominates the noise. Part of the SimCLR advantage over pixels and PCA is attributable to the CNN architecture rather than pretraining: the random frozen encoder beats PCA at 50 and 200 labels and pixels at 200 labels (Table IV), but not clearly pixels at 50 ($p = 0.12$) or either at 10, where its variance is large.

VI. Ablations and Analysis

All ablations change one setting of the SimCLR-style pretraining and keep the probe, splits and seeds fixed; they are run on the same five seeds. The full method (both augmentation families, $\tau = 0.5$, 60 epochs) is the SimCLR-style probe row of Table I.

H4 (augmentations matter): partly supported. At 50 labels the full augmentation reaches 0.955 (Table I), geometric-only 0.927 and photometric-only 0.918 (Table V); the drops are 0.028 (geometric-only; paired $p = 0.09$, not resolved with five seeds) and 0.037 (photometric-only; $p = 0.003$) (Table IV). Only the photometric drop is clearly resolved, so H4 is supported for photometric-only and inconclusive for geometric-only. At 10 labels the

TABLE IV

Additional paired-by-seed comparisons from rh compare on the run registry (A minus B, 5 seeds). p -values are uncorrected: 24 tests are shown here and 15 in Table II, and the three budgets share the same pretrained encoder and test split per seed, so they are not independent replications.

Task	A – B	Δ acc	paired p
n10	SimCLR-style probe – SimCLR geom-only	0.101	0.064
n50	SimCLR-style probe – SimCLR geom-only	0.028	0.090
n200	SimCLR-style probe – SimCLR geom-only	0.003	0.327
n10	SimCLR-style probe – SimCLR photo-only	0.124	0.001
n50	SimCLR-style probe – SimCLR photo-only	0.037	0.003
n200	SimCLR-style probe – SimCLR photo-only	0.013	0.016
n10	Supervised + aug – Supervised scratch	0.051	0.121
n50	Supervised + aug – Supervised scratch	0.004	0.701
n200	Supervised + aug – Supervised scratch	−0.008	0.095
n10	Supervised + aug – SimCLR-style probe	−0.113	0.022
n50	Supervised + aug – SimCLR-style probe	−0.032	0.086
n200	Supervised + aug – SimCLR-style probe	−0.010	0.113
n10	Rotation (reimplemented) – PCA + LR	−0.064	0.050
n50	Rotation (reimplemented) – PCA + LR	−0.048	0.095
n200	Rotation (reimplemented) – PCA + LR	−0.006	0.348
n10	Rotation (reimplemented) – Random CNN + probe	−0.106	0.090
n50	Rotation (reimplemented) – Random CNN + probe	−0.084	0.020
n200	Rotation (reimplemented) – Random CNN + probe	−0.036	0.004
n10	Random CNN + probe – PCA + LR	0.041	0.406
n50	Random CNN + probe – PCA + LR	0.036	0.019
n200	Random CNN + probe – PCA + LR	0.030	0.002
n10	Random CNN + probe – Pixels + LR	−0.017	0.655
n50	Random CNN + probe – Pixels + LR	0.026	0.117
n200	Random CNN + probe – Pixels + LR	0.012	0.013

TABLE V

Augmentation ablation: test accuracy of SimCLR with one augmentation family only (5 seeds).

Method	Task	test_acc $\uparrow$	n
SimCLR geom-only	n50	0.927 ± 0.018	5
SimCLR photo-only	n50	0.918 ± 0.012	5
SimCLR geom-only	n10	0.725 ± 0.116	5
SimCLR photo-only	n10	0.702 ± 0.063	5
SimCLR geom-only	n200	0.975 ± 0.010	5
SimCLR photo-only	n200	0.965 ± 0.016	5

TABLE VI

Supervised scratch with the full SimCLR augmentation applied to the labeled examples (5 seeds).

Method	Task	test_acc $\uparrow$	n
Supervised + aug	n50	0.923 ± 0.033	5
Supervised + aug	n10	0.713 ± 0.099	5
Supervised + aug	n200	0.969 ± 0.014	5

ordering is the same, but the seed std of geometric-only is 0.116, so that comparison is inconclusive. Geometric-only is not distinguishable from photometric-only.

Is the gain just augmentation? Table VI gives scratch training with the same augmentations. It is not distin-

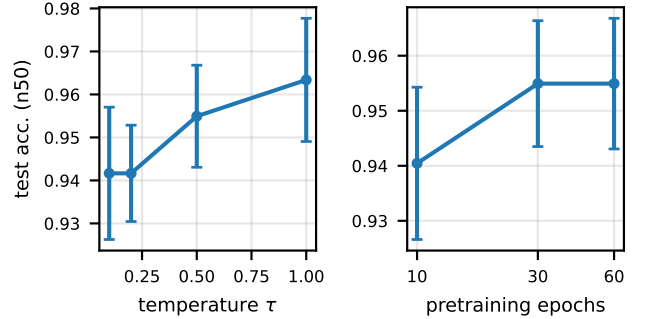


Fig. 3. Sensitivity at 50 labels: temperature τ (left) and pretraining epochs (right); mean $\pm$ std over 5 seeds, including the defaults from the main runs.

guishable from plain scratch at any budget (10 labels: 0.713 vs 0.662, std 0.099, $p = 0.12$) and remains below SimCLR at 10 labels ($p = 0.02$; Table IV). So in this setup augmentation of the labeled examples alone does not reproduce the SimCLR advantage.

Sensitivity. A higher temperature ($\tau = 1.0$: 0.963) is slightly better than the default (0.955) and lower values (0.942 and 0.942) are worse (Table VII, Fig. 3); the differences are about one seed standard deviation, so we make no claim about the optimum; the $\tau = 1.0$ comparison is post hoc on the test split, and the default was not

TABLE VII

Temperature and epoch sweeps at 50 labels (5 seeds). Defaults, $\tau = 0.5$ and 60 epochs, are in Table I.

Method	test_acc $\uparrow$	n
SimCLR tau=0.1	0.942 ± 0.015	5
SimCLR tau=0.2	0.942 ± 0.011	5
SimCLR tau=1.0	0.963 ± 0.014	5

Method	test_acc $\uparrow$	n
SimCLR ep=10	0.940 ± 0.014	5
SimCLR ep=30	0.955 ± 0.011	5

tuned on it. Shortening pretraining to 10 epochs gives 0.940 against 0.955 at 60; 30 epochs matches the default (0.955), so accuracy saturates early at this scale.

VII. Limitations

This is a small CPU study with a single dataset of 8×8 images, a three-layer CNN and five seeds, so p -values are indicative and effects of about one standard deviation are not resolved. Baselines are reimplemented by us, not taken from the original code, and received no tuning: the supervised baseline uses a fixed number of steps without early stopping or validation, and the rotation baseline may be weak partly because of choices we did not explore. The test split is re-drawn per seed, so seeds are not independent samples of the population; the labeled subsets at $n = 10$ (one image per class) make that budget very noisy. The sweeps and ablations are reported on the test split and were not used to choose the headline configuration, but a reader should treat any “best” value in them as post hoc. The unlabeled pool includes the labeled images (without labels), as is common in semi-supervised protocols. The PCA baseline has a fixed dimension (16) and standardised features, was not tuned, and is below raw pixels + LR at every budget, so pixels + LR is the stronger classical baseline here; the PCA gaps are thus likely inflated. We did not test other SSL methods (MoCo, BYOL), other probes, fine-tuning of the pretrained encoder, or larger images; a full study would add them with more seeds, a validation protocol and tuned baselines. Registry wall-clock times are inflated by machine contention. Finally, the gap to PCA may partly reflect the convolutional architecture rather than SSL, as the random-encoder control suggests.

VIII. Conclusion

On 8×8 digits, a SimCLR-style pretrained encoder with a linear probe was the best system at all three label budgets, with a large margin at 10 labels, a smaller one at 50, and no resolvable margin over supervised scratch at 200. Rotation prediction did not help and was below a random encoder. Both augmentation families contribute,

and the temperature and epoch settings matter at the scale of a standard deviation. These findings hold for this dataset, architecture and five-seed budget only.

References

- [1] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” arXiv preprint arXiv:2002.05709, 2020. [Online]. Available: <https://arxiv.org/abs/2002.05709>
- [2] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum Contrast for Unsupervised Visual Representation Learning,” arXiv preprint arXiv:1911.05722, 2019. [Online]. Available: <https://arxiv.org/abs/1911.05722>
- [3] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A. Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko, “Bootstrap your own latent: A new approach to self-supervised Learning,” arXiv preprint arXiv:2006.07733, 2020. [Online]. Available: <https://arxiv.org/abs/2006.07733>
- [4] S. Gidaris, P. Singh, and N. Komodakis, “Unsupervised Representation Learning by Predicting Image Rotations,” arXiv preprint arXiv:1803.07728, 2018. [Online]. Available: <https://arxiv.org/abs/1803.07728>
- [5] X. Zhai, A. Oliver, A. Kolesnikov, and L. Beyer, “S4L: Self-Supervised Semi-Supervised Learning,” arXiv preprint arXiv:1905.03670, 2019. [Online]. Available: <https://arxiv.org/abs/1905.03670>
- [6] T. Chen, S. Kornblith, K. Swersky, M. Norouzi, and G. Hinton, “Big Self-Supervised Models are Strong Semi-Supervised Learners,” arXiv preprint arXiv:2006.10029, 2020. [Online]. Available: <https://arxiv.org/abs/2006.10029>
- [7] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” arXiv preprint arXiv:1807.03748, 2018. [Online]. Available: <https://arxiv.org/abs/1807.03748>
- [8] A. Kolesnikov, X. Zhai, and L. Beyer, “Revisiting Self-Supervised Visual Representation Learning,” arXiv preprint arXiv:1901.09005, 2019. [Online]. Available: <https://arxiv.org/abs/1901.09005>
- [9] J.-C. Su, S. Maji, and B. Hariharan, “Boosting Supervision with Self-Supervision for Few-shot Learning,” arXiv preprint arXiv:1906.07079, 2019. [Online]. Available: <https://arxiv.org/abs/1906.07079>
- [10] S. Konstantakos, J. Cani, I. Mademlis, D. I. Chalkiadaki, Y. M. Asano, E. Gavves, and G. T. Papadopoulos, “Self-supervised visual learning in the low-data regime: a comparative evaluation,” arXiv preprint arXiv:2404.17202, 2024. [Online]. Available: <https://arxiv.org/abs/2404.17202>
- [11] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv preprint arXiv:1412.6980, 2014. [Online]. Available: <https://arxiv.org/abs/1412.6980>